



CS344: Operating Systems Lab

Lab Quiz, October 23, 2024

Set - A

Indian Institute of Technology Guwahati

Duration: 40 Minutes

Total : 20

Marks

Name: _____

Roll No.: _____

Student Signature: _____

Invigilator Signature: _____

(10 Questions)

Part - A

[10 Marks]

> Please mention only the option you select inside the box in capital letters.

Q1. When implementing lazy memory allocation in xv6, how does the kernel trick a process into believing it has the requested memory?

- a> The kernel immediately allocates physical memory but hides this from the process.
- b> The sbrk() system call increases the process's virtual memory size without allocating physical memory.
- c> The kernel swaps out pages to free up memory for the process.
- d> The kernel marks the requested memory as inaccessible until the process accesses it.

Q2. Which of the following steps is NOT required when implementing a getNumProc() system call in the xv6 operating system?

- a> Modifying the system call table to include getNumProc().
- b> Writing a user-level program to invoke getNumProc().
- c> Modifying the proc structure to track the number of active processes.
- d> Implementing the logic in the kernel to count active processes across all states.

Q3. What is the primary purpose of the panic() function in the xv6 operating system kernel?

- a>** To handle system errors by trying to recover and continue execution.
- b>** To log an error message and halt the system immediately.
- c>** To reset the kernel and restart the operating system.
- d>** To send a warning to the user space and allow the process to terminate gracefully.

Q4. In which scenario will a process be marked as SLEEPING?

- a>** When it is running.
- b>** When it is waiting for I/O.
- c>** When it is ready to run.
- d>** When it has terminated.

Q5. When does a PC switch from 16-bit real mode to 32-bit protected mode during booting?

- a>** As soon as the BIOS initializes.
- b>** When the boot loader reads the kernel.
- c>** After the A20 address line is configured.
- d>** Once the user-level program starts executing.

Q6. What happens when the kernel tries to allocate memory for a process and there is no free physical memory available in the xv6 swapping mechanism?

- a> The process is immediately terminated by the kernel.
- b> The kernel swaps out pages to free memory and suspends the requesting process.
- c> The kernel reduces the process's memory allocation to fit available memory.
- d> The process enters a waiting state until other processes release memory.

Q7. What happens if the `sys_draw` system call fails because the buffer is too small?

- a> The system call will retry with a larger buffer.
- b> The system call returns a negative value.
- c> The system crashes.
- d> The system call returns the number of bytes copied.

Q8. What will happen if a user forgets to provide a command-line argument to the `sleep` program?

- a> The program will use a default value of 10 ticks.
- b> The program will terminate without any output.
- c> The program will print an error message.
- d> The program will pause indefinitely.

Q9. If the time complexity of worst case Round Robin Scheduling Algorithm is $O(n^2)$, what is the best case time complexity of Shortest Job First?

- a> $O(1)$.
- b> $O(n)$.
- c> $O(n \log n)$.
- d> $O(\log n)$.

Q10. Which of the following steps is essential when adding a new system call to xv6?

- a>** Modify trap.c to handle the new system call.
- b>** Modify syscall.h to add a new system call number.
- c>** Add the system call to kalloc.c to allocate memory for the process.
- d>** Add the system call to console.c to enable debugging output.

10 Questions

Part-B

[10 Marks]

Q1. How does the bootloader know where the kernel's first instruction is located?

- a>** It is specified in the BIOS settings.
- b>** It reads the e_entry field from the ELF header.
- c>** It calculated the address based on memory size.
- d>** It is hard-coded in the bootloader.

Q2. When implementing lazy memory allocation in xv6,how can the kernel identify the virtual address that caused a page fault, and how should it handle the faulting address during the allocation process?

- a>** By checking the tf->trapno for a value of T_PGFLT and directly allocating memory at the next available page boundary.
- b>** By using tf->trapno to check for a T_PGFLT fault and using PGROUNDDOWN(va) to round the virtual address down to the start of the page.
- c>** By inspecting the proc->sz field and incrementing the address by the size of the page table entry.
- d>** By checking if the faulting address is within kernel space and using allocvm() to allocate memory at the exact virtual address.

Q3. In xv6, which combination of page table entry (PTE) flags would allow a process to execute code but prevent it from writing to memory, and how does this contribute to system stability?

- a> PTE_P | PTE_W.
- b> PTE_P | PTE_U.
- c> PTE_P | PTE_U | PTE_W.
- d> PTE_P | PTE_U | ~PTE_W.

Q4. When the PC is first powered on, at which physical address does the processor start executing instructions?

- a> 0x00000000.
- b> 0x000FFFF0.
- c> 0x000A0000.
- d> 0xF0000000.

Q5. Consider a hybrid [Shortest Job First (SJF) + Round Robin] scheduling algorithm and four processes P1, P2, P3, and P4. The processes come to the Ready Queue at 0- time interval. The burst time for the processes P1, P2, P3, P4 are 10, 12, 8 and 5 respectively. The quantum of time for scheduling is fixed as the burst time of the primary process in the Ready Queue. What is the order in which the processes are set up in the Ready Queue and the time quantum?

- a> P1, P2, P3, P5 and 8-time units.
- b> P2, P3, P1, P4 and 5-time units.
- c> P4, P3, P1, P2 and 5-time units.
- d> P2, P1, P3, P4 and 12-time units.

Q6. What is the role of `usys.s` in `xv6`?

- a> It contains the user-level implementation of system calls.
- b> It provides the definitions for system call interfaces in the kernel.
- c> It contains the assembly code that transitions between user space and kernel space for system calls.
- d> It holds the process control block (PCB) structure for each process.

Q7. How does the x86 architecture handle the translation of large memory spaces (above 4 GB) using its page table structure, and what specific limitations does this impose?

- a> By using an 8-bit page table, which limits the memory space to 256 MB.
- b> By using a 16-bit page directory, which extends the physical memory limit beyond 8 GB.
- c> By using a two-level page table system where the top 10 bits of the virtual address select the page directory, limiting the accessible memory.
- d> By using a two-level page table system where the last 10 bits of the virtual address select the page directory, limiting the accessible memory.

Q8. In real mode (the mode that PC starts off in, before `xv6` OS is booted), how are addresses calculated from CS (segment) and IP (offset) registers ?

- a> $(16 \times \text{segment}) + \text{offset}$.
- b> $\text{segment} + (\text{offset} \ll 4)$.
- c> $(\text{segment} \times \text{offset}) + 100$.
- d> $(\text{segment} \mid 0xFF00) + (\text{offset} \& 0xF0A0)$.

Q9. What happens when an application in user mode attempts to execute a privileged instruction?

- a>** The processor executes the instruction normally.
- b>** The processor ignores the instruction.
- c>** The processor switches to kernel mode for handling.
- d>** The processor immediately halts the system.

Q10. You have implemented a lottery scheduler in xv6, and you observe that a process with 30 tickets is scheduled for 20 ticks over 100 scheduling events. Assuming no bias in randomness, what could be the issue with your implementation?

- a>** The total number of tickets in the system is too low.
- b>** The random number generator has a small range.
- c>** The process is not being added to the scheduling queue properly.
- d>** The system is too slow to generate enough ticks.